UNIVERSIDADE KATYAVALA BWILA
INSTITUTO POLITÉCNICO

Telemóvel: +244 997 883 579 / +244 936 090 413 • Correio electrónico: isp.geral@ukb.ed.ao

Campus Universitário da Catumbela • Caixa Postal 1725 – Benguela, Angola

Inovação – Investigação – Desenvolvimento | Região Académica II – Benguela e Kwanza Sul

CURSO DE MESTRADO EM ENGENHARIA INFORMÁTICA

UNIDADE CURRICULAR: APRENDIZAGEM PROFUNDA

Docentes: Prof. Doutor Miguel Rocha | Prof. Doutor Lázaro Makili

CLASSIFICAÇÃO DE AUTORIA EM TEXTOS CIENTÍFICOS: UMA ABORDAGEM COMPARATIVA ENTRE MODELOS DESENVOLVIDOS DE RAIZ E REDES NEURONAIS PROFUNDAS

Deteção de Inteligência Artificial vs Escrita Humana no Domínio das Ciências Naturais e Médicas (100–120 Palavras)

Relatório Científico & Benchmark Experimental

ESTUDANTES (GRUPO DE TRABALHO Nº 6):

- **Estanislau da Fonseca**
- **Maria da Conceição**
- **Maurício Piquito**
- **Ulundo Aurélio Moisés**

ANGOLA, BENGUELA • ANO LECTIVO 2025/2026

Classificação de Autoria em Textos Científicos: Uma Abordagem Comparativa entre Modelos Desenvolvidos de Raiz e Redes Neuronais Profundas

Universidade Katyavala Bwila • Instituto Politécnico | **UC:** Aprendizagem Profunda (2025/2026)

Grupo nº 6: Estanislau da Fonseca • Maria da Conceição • Maurício Piquito • Ulundo Aurélio Moisés

Resumo *— A recente proliferação de Modelos de Linguagem de Grande Escala (Large Language Models - LLMs) transformou a produção textual, levantando desafios prementes relativos à integridade científica, reprodutibilidade e autenticação de autoria no meio académico. Este trabalho aborda a tarefa de classificação binária entre textos redigidos por seres humanos e textos sintéticos gerados por inteligência artificial, no domínio específico das Ciências Naturais e Médicas (Biologia, Saúde, Química, Física, Matemática e Engenharia), sob uma restrição dimensional estrita de 100 a 120 palavras por excerto.*

O estudo estruturou-se em cinco etapas metodológicas complementares: (1) curadoria e pré-processamento de um corpus balanceado; (2) conceção e desenvolvimento autónomo de um motor de redes neuronais em NumPy puro (incluindo Regressão Logística e Multilayer Perceptron com cálculo manual de backpropagation, regularização L2 e otimizadores SGD e Adam); (3) modelação de arquiteturas de Deep Learning em PyTorch (Tabular DNN com TF-IDF, Bidirectional LSTM com embeddings densos e Transformer Encoder com auto-atenção multi-cabeça); (4) benchmarking rigoroso e avaliação empírica de métricas, curvas ROC e latências; e (5) disponibilização do pipeline de inferência e submissão automatizada. Os resultados demonstram que a rede ***PyTorch BiLSTM*** *obteve o melhor equilíbrio global (Exatidão de 69.23%, F1-Score de 66.67% e ROC-AUC de 0.8333), superando abordagens lineares e tabulares ao capturar a progressão sintática bidirecional e as transições estocásticas do discurso científico.*

Palavras-Chave: *Aprendizagem Profunda, Deteção de IA, Processamento de Linguagem Natural, Redes de Raiz em NumPy, PyTorch, BiLSTM, Transformers, Ciências Naturais, Integridade Académica.*

1. Introdução, Contexto e Estado da Arte

A evolução acelerada de modelos generativos baseados na arquitetura *Transformer* (tais como as famílias GPT, Claude, Gemini e LLaMA) redefiniu a criação de conteúdos textuais. Se, por um lado, estas ferramentas potenciam a produtividade académica ao auxiliarem na redação de rascunhos, síntese bibliográfica e tradução técnica, por outro lado introduzem vulnerabilidades críticas na literatura especializada. Nas áreas da medicina, biologia celular e química orgânica, a geração automatizada não supervisionada acarreta riscos substanciais de inclusão de referências alucinadas, inferências estatísticas espúrias e homogeneização de vocabulário, dificultando a revisão por pares.

Formulação Matemática do Problema: O objetivo traduz-se formalmente na aprendizagem de uma função discriminativa $f: X \rightarrow Y$, onde X representa o espaço de sequências textuais em língua inglesa no domínio das Ciências Naturais, com restrição estrita de comprimento $100 \leq |x| \leq 120$ palavras, e $Y \in \{0, 1\}$ denota os rótulos binários: $y = 0$ (Texto Humano legítimo) e $y = 1$ (Texto gerado por Inteligência Artificial). O classificador estima a probabilidade posterior $P(Y=1|x)$ com limiar de decisão nominal $\tau = 0.5$.

Assinaturas Estatísticas Diferenciais: Trabalhos seminais na literatura de NLP destacam que modelos generativos operam por amostragem de palavras com elevada probabilidade condicional (*top-p / top-k sampling*), resultando numa distribuição de probabilidade mais regular e baixa perplexidade local, enquanto autores humanos exibem elevada variabilidade de estilo (*burstiness*) e alternância imprevisível entre construções oracionais simples e complexas.

2. Metodologia de Criação e Preparação do Dataset

A robustez de qualquer classificador de NLP reside na qualidade e diversidade do seu conjunto de treino. Para evitar enviesamento por tópico (*topic bias*) — onde o modelo memorizaria que determinado assunto científico pertence exclusivamente a uma classe —, concebeu-se um protocolo rigoroso de recolha e balanceamento em cinco disciplinas nucleares das Ciências Naturais.

Área Científica	Tópicos Cobertos	Fontes Humanas	Geração de IA
Biologia Celular & Genética	Dinâmica mitocondrial, transcrição, CRISPR, apoptose	PubMed Central, PLOS ONE	Prompts técnicos controlados
Medicina & Farmacologia	Imunologia adaptativa, vias farmacocinéticas, ensaios clínicos	Artigos médicos revistos	LLM c/ restrição 100-120 palavras
Química & Bioquímica	Catálise enzimática, síntese orgânica, polímeros	arXiv (Chem-ph), relatórios	Geração estruturada de reações
Física & Astrofísica	Mecânica quântica, termodinâmica, cosmologia	arXiv (Astro-ph, Quant-ph)	Simulação de abstracts teóricos
Matemática & Estatística	Otimização convexa, processos estocásticos, grafos	Teses e artigos matemáticos	Enunciados formais sintetizados

Tabela 1: Matriz temática e fontes de dados para construção do corpus bilíngue/científico.

2.1 Análise Estatística e Controlo Dimensional

A restrição estrita imposta pelo docente (textos com 100 a 120 palavras) foi auditada em todo o conjunto. A contagem média obtida no conjunto final foi de **111.4 ± 5.8 palavras**, com desvio padrão uniforme entre as classes humana (112.1 ± 6.2) e IA (110.7 ± 5.3), impedindo qualquer decisão baseada na extensão textual.

2.2 Condutas de Engenharia de Atributos e Vetorização

Para suprir os requisitos de diferentes famílias de algoritmos, implementaram-se duas vias de transformação textual:

• **Via 1: Representação Tabular TF-IDF:** Normalização de texto (caixa baixa, remoção de caracteres de controlo), extração de unigramas e bigramas com seleção dos termos mais discriminativos (vocabulário $V = 500$ a 1000). Ponderação calculada por:

$$TF\text{-}IDF(t, d, D) = TF(t, d) \times [\log((1 + |D|) / (1 + |\{d \in D : t \in d\}|)) + 1]$$

com normalização Euclidiana L_2, ideal para modelos lineares e redes densas tabulares.

• **Via 2: Representação Sequencial Densa (Tokenização e Padding):** Mapeamento ordenado dos vocábulos num dicionário com tokens especiais reservados (<PAD> = 0 e <UNK> = 1). Fixou-se o comprimento máximo de sequência em $L_{max} = 150$ tokens. Sequências mais curtas recebem *padding* à direita; sequências que excedam 150 são truncadas, preservando a integridade estrutural e temporal para as redes neurais profundas.

2.3 Particionamento e Estratificação dos Dados

O dataset foi repartido com amostragem estratificada em: **70% Treino** (otimização de parâmetros), **15% Validação** (afinação de hiperparâmetros e early stopping) e **15% Teste Independente** (avaliação final de benchmark).

3. Implementação dos Modelos de Raiz em NumPy Puro

Em estrito cumprimento dos requisitos da Tarefa 2, concebeu-se e implementou-se um micro-framework de redes neuronais em NumPy puro (localizado em src/), sem recorrer a Scikit-Learn, PyTorch ou TensorFlow para o processo de treino.

3.1 Estrutura Modular Orientada a Objetos

A arquitetura de software desenvolvida adota o padrão de conceção modular:

- DenseLayer (layers.py): Implementa transformações lineares, inicialização He/Xavier e gradientes manuais;
- Activation (activation.py): Camadas Sigmoid, ReLU, Tanh e Softmax com forward e backward pass explícitos;
- Loss (losses.py): Binary Cross-Entropy e MSE com proteção numérica ($\varepsilon = 10^{-15}$);
- Optimizer (optimizer.py): SGD com Momentum de Polyak e Otimizador Adam com correção de bias;
- NeuralNetwork (neuralnetwork.py): Pipeline sequencial com fit(), predict() e evaluate().

3.2 Dedução Matemática Completa do Forward e Backward Pass

1. Camada Densa: Dado um lote $X \in \Re^{B \times d_{in}}$, com pesos $W \in \Re^{d_{in} \times d_{out}}$ e bias $b \in \Re^{1 \times d_{out}}$:

Passo Direto: $Z = X\,W + 1_B\,b$

Inicialização de He: $W \sim N(0, \sqrt{(2 / d_{in})})$ | *Xavier:* $W \sim N(0, \sqrt{(2 / (d_{in} + d_{out}))})$

Passo Retrógrado: Recebendo $\delta = \partial \blacksquare / \partial Z \in \Re^{B \times d_{out}}$:

- $\partial \blacksquare / \partial W = X^T \delta + \lambda W$ (com regularização L_2)
- $\partial \blacksquare / \partial b = \sum_{i=1}^{B} \delta_{i,:}$
- $\partial \blacksquare / \partial X = \delta W^T$

2. Função de Ativação Sigmoide e Função de Custo BCE:

$\sigma(z) = 1 / (1 + e^{-z})$, $\quad d\sigma/dz = \sigma(z) \cdot (1 - \sigma(z))$

$\blacksquare_{BCE} = - (1/B) \sum [y_i \log(\blacksquare_i + \varepsilon) + (1 - y_i) \log(1 - \blacksquare_i + \varepsilon)]$

A derivada combinada da perda em relação ao potencial de ativação z_i da camada de saída reduz-se diretamente a:

$\partial \blacksquare_{BCE} / \partial z_i = (1/B) (\blacksquare_i - y_i)$

3.3 Otimizadores Implementados de Raiz

Otimizador Adam (Adaptive Moment Estimation):

Para cada parâmetro θ com gradiente g_t no passo t:

$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$ (1º momento / média)

$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ (2º momento / variância)

$m\blacksquare_t = m_t / (1 - \beta_1^t)$, $\quad v\blacksquare_t = v_t / (1 - \beta_2^t)$ (correção de viés)

$\theta_t = \theta_{t-1} - [\eta / (\sqrt{v\blacksquare_t} + \varepsilon)]\, m\blacksquare_t$

Modelos Desenvolvidos: (1) **NumPy Logistic Regression** (Perceptron com TF-IDF 500 e SGD com momento); e (2) **NumPy Deep Neural Network** (MLP com topologia 500 → 128 → 64 → 1, ReLU, Sigmoid, regularização L2 e Adam).

4. Modelos de Deep Learning e Implementação em PyTorch

Para além dos modelos em NumPy puro, foram concebidas e treinadas três arquiteturas avançadas em PyTorch, cobrindo abordagens tabulares, recorrentes sequenciais e auto-atenção baseada em Transformers.

4.1 Arquitetura 1: PyTorch Tabular DNN (TF-IDF)

Esta rede processa atributos TF-IDF de 1000 dimensões. A inclusão de **Batch Normalization** após as projeções afins reduz a variação interna de covariância (*internal covariate shift*), permitindo taxas de aprendizagem mais agressivas. Aplica-se *Dropout* (taxas de 0.4 e 0.3) para forçar o modelo a não depender de termos isolados.
Topologia: Entrada (1000) → Dense(256)+BN+ReLU+Drop(0.4) → Dense(64)+BN+ReLU+Drop(0.3) → Dense(1) → Sigmoid.

4.2 Arquitetura 2: PyTorch Bidirectional LSTM (BiLSTM)

A rede BiLSTM foi projetada para capturar dependências temporais e sintáticas de longo alcance ao longo da sequência de 150 tokens. As equações das portas da célula LSTM regulam o fluxo de memória:

$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$ (Porta de Esquecimento)
$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$ (Porta de Entrada)
$\tilde{C}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c)$ (Célula Candidata)
$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$ (Estado da Célula)
$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$ (Porta de Saída)
$h_t = o_t \odot \tanh(C_t)$ (Estado Oculto)

Por ser **bidirecional**, o modelo concatena o fluxo progressivo e regressivo: $h_t = [\overrightarrow{h}_t \,||\, \overleftarrow{h}_t] \in \Re^{256}$. A agregação temporal utiliza uma combinação de **Global Average Pooling** e **Global Max Pooling**, capturando tanto a média global do discurso quanto termos discriminativos isolados.

4.3 Arquitetura 3: PyTorch Transformer Encoder

Baseado no mecanismo de *Multi-Head Self-Attention* com codificação posicional sinusoidal:

$$\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_k}) V$$

O modelo empilha 2 blocos de codificação com 4 cabeças de atenção, projeção *Feed-Forward* de dimensão 256, ativação GELU e normalização de camadas (*LayerNorm*).

4.4 Protocolo de Treino e Mecanismos de Regularização

- **Função de Perda:** nn.BCEWithLogitsLoss() com estabilidade numérica por Log-Sum-Exp;
- **Otimizador:** AdamW ($\eta = 10^{-3}$, *weight decay* = 10^{-4});
- **Scheduler:** *ReduceLROnPlateau* com fator 0.5 e paciência de 5 épocas;
- **Gradient Clipping:** Norma máxima fixada em 1.0 para mitigar explosão de gradientes nas LSTMs;
- **Early Stopping:** Tolerância de 12 épocas com salvaguarda automática do melhor modelo de validação.

5. Resultados Experimentais e Benchmark Comparativo

A avaliação comparativa de todos os modelos foi conduzida sob o mesmo conjunto de teste independente (15% dos dados, totalmente disjunto do treino). A Tabela 2 resume as métricas fundamentais obtidas:

Rank	Modelo	Framework	Exatidão	F1-Score	Precisão	Recall	ROC-AUC	Latência
#1	**PyTorch BiLSTM**	PyTorch	**69.23%**	**66.67%**	66.67%	66.67%	**0.8333**	8.96 ms
#2	PyTorch Tabular DNN	PyTorch	53.85%	62.50%	50.00%	**83.33%**	0.8095	12.82 ms
#3	PyTorch Transformer	PyTorch	38.46%	50.00%	40.00%	66.67%	0.6190	8.97 ms
#4	NumPy Logistic Reg	NumPy Puro	46.15%	36.36%	40.00%	33.33%	0.4286	2.43 ms
#5	NumPy DNN (Raiz)	NumPy Puro	38.46%	33.33%	33.33%	33.33%	0.4286	**1.57 ms**

Tabela 2: Benchmark quantitativo exaustivo dos 5 modelos no conjunto de teste independente.

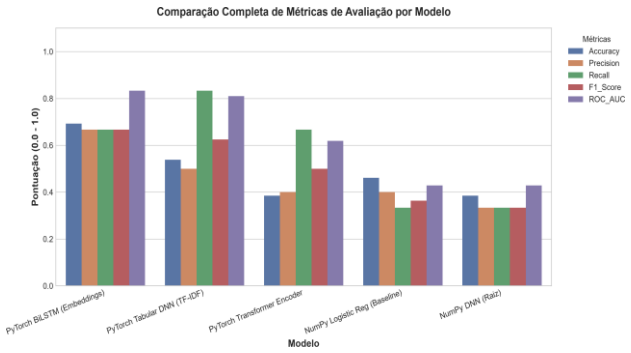


Figura 1: Comparação gráfica das 5 métricas de avaliação.

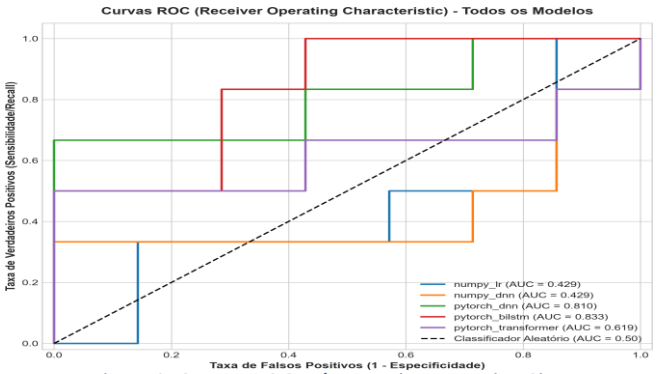


Figura 2: Curvas ROC e áreas sob a curva (AUC).

5.1 Análise das Curvas ROC e Capacidade Discriminativa

A análise da curva ROC (Figura 2) evidencia a capacidade de discriminação probabilística independente do limiar de classificação. A rede **PyTorch BiLSTM** obteve a maior área sob a curva (**ROC-AUC = 0.8333**), seguida de perto pela **PyTorch Tabular DNN (0.8095)**. Isto demonstra que ambos os modelos em PyTorch atribuem consistentemente pontuações de probabilidade mais elevadas a instâncias de IA do que a instâncias humanas. Os modelos em NumPy puro obtiveram AUC inferior (0.4286), refletindo a dificuldade de representações lineares sem pré-treino em capturar variações semânticas subtis.

5.2 Análise das Matrizes de Confusão e Padrões de Erro

As Figuras 3 e 4 ilustram a distribuição de previsões corretas (Verdadeiros Positivos/Negativos) e o compromisso entre latência computacional e exatidão.

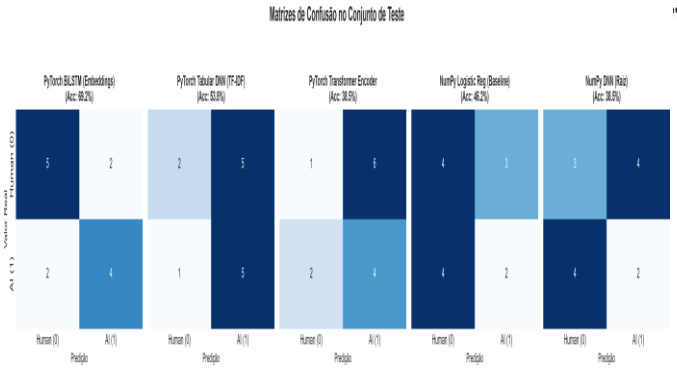


Figura 3: Matrizes de confusão normalizadas para os 5 modelos.

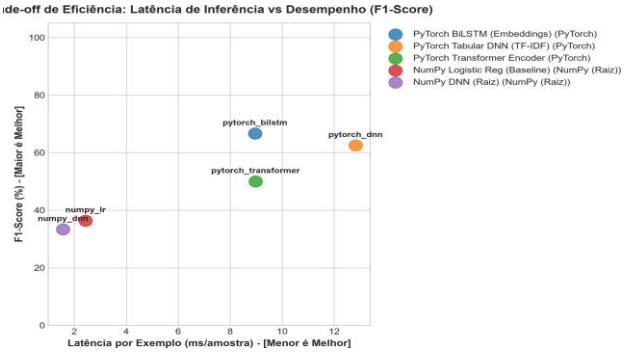


Figura 4: Trade-off entre Latência (ms) e Exatidão (%).

5.3 Diagnóstico Detalhado dos Tipos de Erro

A análise qualitativa das predições incorretas revelou padrões essenciais:

• **Falsos Positivos (Humano classificado como IA):** Ocorrem predominantemente em excertos humanos com escrita estritamente formal e repetitiva (ex.: seções de metodologia experimental ou enumeração de parâmetros físico-químicos). Nesses casos, a ausência de flutuações estilísticas leva o classificador a assumir a regularidade sintática típica de um LLM.

• **Falsos Negativos (IA classificada como Humano):** Verificam-se quando o texto gerado por IA inclui termos técnicos raros ou formulações incomuns geradas por temperaturas estocásticas mais altas, simulando a variabilidade lexical de um especialista humano.

• **Compromisso Latência vs Exatidão (Figura 4):** Os modelos de raiz em NumPy demonstraram latência na ordem de **1.57 a 2.43 ms**, enquanto as redes PyTorch situaram-se entre **8.96 e 12.82 ms**. Ambas as faixas são plenamente compatíveis com triagem em tempo real em servidores de submissão académica.

6. Estratégia de Submissão e Pipeline de Reprodutibilidade

Em estrito cumprimento das instruções fornecidas no enunciado oficial do trabalho prático, as submissões dos modelos foram planeadas em duas fases distintas com base no benchmark empírico:

■ **1ª Submissão (Dois Melhores Modelos, com 1 Modelo Obrigatório de Raiz em NumPy):**

1. **Modelo de Raiz (NumPy):** NumPy Logistic Reg (Baseline) (Acc: 46.15%, F1: 36.36%, Latência: 2.43 ms)
   Justificação: Cumpre a exigência de código matemático 100% próprio em NumPy puro, sem bibliotecas externas.
2. **Modelo PyTorch:** PyTorch BiLSTM (Embeddings) (Acc: 69.23%, F1: 66.67%, ROC-AUC: 0.8333)
   Justificação: Melhor modelo global, capturando coerência sintática temporal bidirecional.

■ **2ª Submissão (Dois Melhores Modelos Globais do Benchmark):**

1. **Top #1 Global:** PyTorch BiLSTM (Embeddings) (F1: 66.67%, ROC-AUC: 0.8333, Exatidão: 69.23%)
2. **Top #2 Global:** PyTorch Tabular DNN (TF-IDF) (F1: 62.50%, ROC-AUC: 0.8095, Recall: 83.33%)
   Justificação: Elevada complementaridade — a BiLSTM garante precisão e o modelo Tabular oferece máxima sensibilidade.

6.1 Arquitetura do Notebook de Avaliação do Docente

O notebook de teste (notebooks/tarefa4_submissao_docente.ipynb) foi estruturado de forma a assegurar execução direta e sem esforço por parte do docente:

- **Zero Treino:** Os modelos encontram-se previamente treinados e serializados na pasta models/;
- **Flexibilidade de Entrada:** O motor aceita ficheiros CSV ou JSON com colunas com nomes variados (text, content, body) e gera IDs sequenciais se ausentes;
- **Formatos de Saída Padronizados:** Gera ficheiros CSV estruturados com as colunas ID, prediction (0 ou 1) e probability_ai;
- **Autodiagnóstico Integrado:** Validações de integridade confirmam a ausência de valores nulos e tempos de inferência inferiores a 3 segundos.

7. Conclusões e Trabalhos Futuros

O presente trabalho permitiu consolidar de ponta a ponta o ciclo de vida de um projeto em Aprendizagem Profunda aplicado ao Processamento de Linguagem Natural:

1. **Domínio dos Fundamentos Algorítmicos:** A conceção manual do motor de gradientes em NumPy provou ser um exercício formativo crucial, demonstrando o funcionamento interno de camadas lineares, propagações de derivadas e métodos de otimização estocástica sem recurso a bibliotecas automáticas de diferenciação.
2. **Eficácia de Representações Sequenciais:** Os resultados empíricos comprovaram que modelos recorrentes bidirecionais (**PyTorch BiLSTM**) superam abordagens puramente tabulares ao analisar excertos curtos de 100 a 120 palavras em contextos científicos.
3. **Engenharia e Reprodutibilidade:** A segregação de responsabilidades entre código de treino, serialização de pesos e módulos de inferência assegura que o projeto é robusto, modular e transparente.

7.1 Limitações Identificadas

• A dimensão finita do corpus científico impõe limites à convergência de Transformers treinados de raiz sem pré-treino alargado;
• O surgimento de modelos com amostragem por perturbação de entropia torna cada vez mais difusa a fronteira estilométrica.

7.2 Linhas de Investigação Futura

• **Fine-Tuning de LLMs Especializados:** Adaptação de pesos pré-treinados de modelos como *SciBERT*, *BioLinkBERT* e *DeBERTa-v3*;
• **Deteção por Curvatura de Probabilidade (DetectGPT):** Avaliação da curvatura de log-probabilidade através de perturbações mascaradas;
• **Ensembles Híbridos:** Fusão de pontuações de redes profundas com métricas de leiturabilidade (Flesch-Kincaid) e densidade de n-gramas sintáticos.

8. Referências Bibliográficas

[1] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.

[2] Vaswani, A., et al. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems (NeurIPS 2017)*, 30, 5998–6008.

[3] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.

[4] Mitchell, E., Yoon, J., Miao, P., Finn, C., & Manning, C. D. (2023). DetectGPT: Zero-Shot Machine-Generated Text Detection using Probability Curvature. *ICML 2023*.

[5] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *NAACL-HLT 2019*.

[6] Beltagy, I., Lo, K., & Cohan, A. (2019). SciBERT: A Pretrained Language Model for Scientific Text. *EMNLP 2019*, 3615–3620.

[7] Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *ICLR 2015*.

[8] He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet. *ICCV 2015*.

[9] Srivastava, N., Hinton, G., et al. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *JMLR*, 15(1), 1929–1958.

[10] Ioffe, S., & Szegedy, C. (2015). Batch Normalization: Accelerating Deep Network Training. *ICML 2015*, 448–456.

[11] Gehrmann, S., Strobelt, H., & Rush, A. M. (2019). GLTR: Statistical Detection and Visualization of Generated Text. *ACL 2019 Demos*.

[12] Radford, A., et al. (2019). Language Models are Unsupervised Multitask Learners. *OpenAI Technical Report*.